Capítulo 10 - Agente de Execução (Execution Agent)

10.1. Introdução ao Agente de Execução

O **Agente de Execução** é o componente operacional do Projeto Quimera. Ele recebe mensagens aprovadas do Agente Escriba e **as envia de forma automatizada** para os perfis qualificados, simulando comportamento humano para evitar detecção.

10.1.1. Responsabilidades do Agente de Execução

1. **Receber mensagens aprovadas:**
   - Escutar fila de mensagens prontas para envio
   - Verificar se perfil ainda está qualificado
2. **Navegar até perfil:**
   - Abrir Instagram/Facebook
   - Navegar até perfil do destinatário
   - Verificar se perfil ainda é público
3. **Enviar mensagem:**
   - Clicar em "Enviar mensagem"
   - Digitar mensagem (simulando digitação humana)
   - Enviar
4. **Registrar envio:**
   - Atualizar status no banco de dados
   - Criar registro de conversa
   - Emitir evento MensagemEnviada
5. **Respeitar limites:**
   - Rate limiting (máximo X mensagens por hora)
   - Delays entre ações
   - Pausas aleatórias

10.1.2. Desafios Principais

1. Detecção de Automação

Instagram detecta bots que enviam mensagens em massa. Precisamos simular comportamento humano perfeitamente.

2. Limites de Mensagens

Instagram limita mensagens diretas:

- ~20 mensagens por hora
- ~50 mensagens por dia
- Contas novas têm limites menores

3. Captchas e Verificações

Instagram pode exigir verificação (captcha, código SMS) se detectar atividade suspeita.

4. Mudanças na Interface

Botões e elementos HTML mudam frequentemente.

10.2. Arquitetura do Agente de Execução

10.2.1. Componentes Principais

Plain Text

```
┌──────────────────────────────────────────────────────────┐
│            AGENTE DE EXECUÇÃO                            │
├──────────────────────────────────────────────────────────┤
│                                                          │
│   ┌──────────────────────────────────────┐               │
│   │  1. GERENCIADOR DE FILAS             │               │
│   │  - Escuta fila de mensagens          │               │
│   │  - Prioriza por pontuação            │               │
│   │  - Verifica rate limits              │               │
│   └──────────────────────────────────────┘               │
│                     │                                     │
│                     ▼                                     │
│   ┌──────────────────────────────────────┐               │
│   │  2. GERENCIADOR DE SESSÕES           │               │
│   │  - Mantém sessão ativa               │               │
│   │  - Rotação de contas                 │               │
│   │  - Rotação de proxies                │               │
│   └──────────────────────────────────────┘               │
│                     │                                     │
│                     ▼                                     │
│   ┌──────────────────────────────────────┐               │
│   │  3. NAVEGADOR AUTOMATIZADO           │               │
```

```
│   │  - Selenium/Playwright              │              │
│   │  - Simula comportamento humano      │              │
│   │  - Executa ações (navegar, clicar)  │              │
│   └─────────────────────────────────────┘              │
│                    │                                    │
│                    ▼                                    │
│   ┌─────────────────────────────────────┐              │
│   │  4. SIMULADOR DE DIGITAÇÃO          │              │
│   │  - Digita caractere por caractere   │              │
│   │  - Velocidade variável              │              │
│   │  - Erros e correções ocasionais     │              │
│   └─────────────────────────────────────┘              │
│                    │                                    │
│                    ▼                                    │
│   ┌─────────────────────────────────────┐              │
│   │  5. DETECTOR DE ERROS               │              │
│   │  - Detecta captchas                 │              │
│   │  - Detecta bloqueios                │              │
│   │  - Detecta erros de rede            │              │
│   └─────────────────────────────────────┘              │
│                    │                                    │
│                    ▼                                    │
│   ┌─────────────────────────────────────┐              │
│   │  6. PERSISTÊNCIA                    │              │
│   │  - Atualiza status no BD            │              │
│   │  - Cria registro de conversa        │              │
│   │  - Emite evento MensagemEnviada     │              │
│   └─────────────────────────────────────┘              │
│                                                        │
└────────────────────────────────────────────────────────┘
```

10.2.2. Fluxo de Execução

Plain Text

```
INÍCIO
  │
  ├─> Escutar fila: mensagens_para_envio
  │
  ├─> Para cada mensagem:
  │     │
  │     ├─> 1. VERIFICAR RATE LIMITS
  │     │     ├─> Verificar mensagens enviadas na última hora
  │     │     ├─> Se limite atingido: aguardar
  │     │     └─> Se OK: continuar
  │     │
  │     ├─> 2. INICIALIZAR SESSÃO
```

```
│      │      ├─> Verificar se sessão ativa existe
│      │      ├─> Se não: criar nova sessão
│      │      │     ├─> Selecionar conta
│      │      │     ├─> Selecionar proxy
│      │      │     ├─> Abrir navegador
│      │      │     └─> Fazer login (se necessário)
│      │      └─> Se sim: reutilizar
│      │
│      ├─> 3. NAVEGAR ATÉ PERFIL
│      │      ├─> Abrir URL do perfil
│      │      ├─> Aguardar carregamento (2-5s)
│      │      ├─> Verificar se perfil existe
│      │      ├─> Verificar se perfil é público
│      │      └─> Scroll leve (simular leitura)
│      │
│      ├─> 4. ABRIR CHAT
│      │      ├─> Localizar botão "Enviar mensagem"
│      │      ├─> Mover mouse até botão
│      │      ├─> Aguardar (0.5-1.5s)
│      │      ├─> Clicar no botão
│      │      ├─> Aguardar modal abrir (1-3s)
│      │      └─> Verificar se modal abriu
│      │
│      ├─> 5. DIGITAR MENSAGEM
│      │      ├─> Localizar campo de texto
│      │      ├─> Clicar no campo
│      │      ├─> Aguardar (0.3-0.8s)
│      │      ├─> Digitar caractere por caractere
│      │      │     ├─> Velocidade: 80-150ms por caractere
│      │      │     ├─> Pausas aleatórias em espaços
│      │      │     └─> Ocasionalmente: erro + backspace
│      │      └─> Aguardar (1-2s) após terminar
│      │
│      ├─> 6. ENVIAR MENSAGEM
│      │      ├─> Localizar botão "Enviar"
│      │      ├─> Aguardar (0.5-1s)
│      │      ├─> Clicar no botão
│      │      ├─> Aguardar confirmação (1-2s)
│      │      └─> Verificar se enviou
│      │
│      ├─> 7. REGISTRAR ENVIO
│      │      ├─> Atualizar status no PostgreSQL
│      │      ├─> Criar registro de conversa
│      │      ├─> Emitir evento MensagemEnviada
│      │      └─> Incrementar contador de rate limit
│      │
│      ├─> 8. AGUARDAR PRÓXIMA
│      │      └─> Delay aleatório (30-120 segundos)
```

```
  |     |
  |     └─> Tratar erros (se houver)
  |           ├─> Captcha: pausar e alertar
  |           ├─> Bloqueio: trocar conta/proxy
  |           └─> Erro de rede: retry
  |
FIM
```

10.3. Gerenciamento de Sessões

10.3.1. Sessões Persistentes

Manter sessão do navegador ativa entre envios reduz tempo e risco de detecção.

Python

```python
import undetected_chromedriver as uc
from selenium.webdriver.common.by import By
import time

class GerenciadorSessao:
    def __init__(self):
        self.driver = None
        self.sessao_ativa = False
        self.timestamp_ultimo_uso = None
        self.mensagens_enviadas = 0

    def inicializar_sessao(self, conta, proxy):
        """Inicializa nova sessão do navegador."""
        if self.driver:
            self.fechar_sessao()

        # Configurar opções
        options = uc.ChromeOptions()
        options.add_argument(f'--proxy-server={proxy}')
        options.add_argument('--disable-blink-features=AutomationControlled')

        # Criar driver
        self.driver = uc.Chrome(options=options)

        # Fazer login
        self.fazer_login(conta)

        self.sessao_ativa = True
        self.timestamp_ultimo_uso = time.time()
        self.mensagens_enviadas = 0
```

```
        return self.driver

    def fazer_login(self, conta):
        """Faz login no Instagram."""
        self.driver.get("https://www.instagram.com/accounts/login/")
        time.sleep(3)

        # Preencher username
        username_input = self.driver.find_element(By.NAME, "username")
        self.digitar_humano(username_input, conta['username'])

        # Preencher senha
        senha_input = self.driver.find_element(By.NAME, "password")
        self.digitar_humano(senha_input, conta['senha'])

        # Clicar em login
        login_button = self.driver.find_element(By.XPATH,
"//button[@type='submit']")
        login_button.click()

        time.sleep(5)

        # Verificar se logou
        if "login" in self.driver.current_url:
            raise Exception("Falha no login")

    def digitar_humano(self, elemento, texto):
        """Digita texto simulando humano."""
        for char in texto:
            elemento.send_keys(char)
            time.sleep(random.uniform(0.08, 0.15))

    def verificar_sessao_valida(self):
        """Verifica se sessão ainda é válida."""
        if not self.sessao_ativa:
            return False

        # Verificar se expirou (30 minutos)
        tempo_inativo = time.time() - self.timestamp_ultimo_uso
        if tempo_inativo > 1800:
            return False

        # Verificar se atingiu limite de mensagens
        if self.mensagens_enviadas >= 20:
            return False

        return True
```

```python
    def atualizar_uso(self):
        """Atualiza timestamp de último uso."""
        self.timestamp_ultimo_uso = time.time()
        self.mensagens_enviadas += 1

    def fechar_sessao(self):
        """Fecha sessão do navegador."""
        if self.driver:
            self.driver.quit()
            self.driver = None

        self.sessao_ativa = False
```

10.3.2. Rotação de Contas

Usar múltiplas contas para distribuir mensagens e evitar limites.

Python

```python
class GerenciadorContas:
    def __init__(self, contas):
        self.contas = contas
        self.indice_atual = 0
        self.uso_por_conta = {conta['username']: 0 for conta in contas}

    def obter_proxima_conta(self):
        """Retorna próxima conta disponível."""
        # Tentar encontrar conta com uso < 20 mensagens hoje
        for _ in range(len(self.contas)):
            conta = self.contas[self.indice_atual]

            if self.uso_por_conta[conta['username']] < 20:
                self.indice_atual = (self.indice_atual + 1) %
len(self.contas)
                return conta

            self.indice_atual = (self.indice_atual + 1) % len(self.contas)

        # Todas as contas atingiram limite
        return None

    def registrar_uso(self, conta):
        """Registra uso de uma conta."""
        self.uso_por_conta[conta['username']] += 1

    def resetar_contadores_diarios(self):
```

```
        """Reseta contadores diários (executar à meia-noite)."""
        self.uso_por_conta = {conta['username']: 0 for conta in self.contas}
```

10.4. Simulação de Comportamento Humano

10.4.1. Digitação Humana

Humanos não digitam em velocidade constante. Simulamos variações.

Python

```python
import random
import time

def digitar_como_humano(elemento, texto):
    """
    Digita texto simulando comportamento humano.

    - Velocidade variável
    - Pausas em espaços e pontuação
    - Ocasionalmente: erros e correções
    """
    for i, char in enumerate(texto):
        # Velocidade base: 80-150ms por caractere
        delay = random.uniform(0.08, 0.15)

        # Pausas maiores em espaços e pontuação
        if char in [' ', ',', '.', '!', '?']:
            delay += random.uniform(0.1, 0.3)

        # Pausas maiores no início de frases
        if i > 0 and texto[i-1] in ['.', '!', '?']:
            delay += random.uniform(0.3, 0.6)

        # Ocasionalmente: erro de digitação (5% de chance)
        if random.random() < 0.05 and char.isalpha():
            # Digitar caractere errado
            char_errado = random.choice('abcdefghijklmnopqrstuvwxyz')
            elemento.send_keys(char_errado)
            time.sleep(random.uniform(0.1, 0.3))

            # Corrigir (backspace + caractere certo)
            elemento.send_keys('\b')
            time.sleep(random.uniform(0.05, 0.15))

        # Digitar caractere correto
```

```
        elemento.send_keys(char)
        time.sleep(delay)

    # Pausa após terminar de digitar
    time.sleep(random.uniform(0.5, 1.5))
```

10.4.2. Movimentos de Mouse

Mover mouse de forma natural, não teleportar.

Python

```python
from selenium.webdriver.common.action_chains import ActionChains
import numpy as np

def mover_mouse_natural(driver, elemento_destino):
    """Move mouse de forma natural até elemento."""
    action = ActionChains(driver)

    # Obter posição do elemento
    localizacao = elemento_destino.location
    tamanho = elemento_destino.size

    # Calcular centro do elemento
    x_destino = localizacao['x'] + tamanho['width'] / 2
    y_destino = localizacao['y'] + tamanho['height'] / 2

    # Gerar curva Bezier para movimento suave
    pontos = gerar_curva_bezier((0, 0), (x_destino, y_destino),
num_pontos=20)

    # Mover mouse pelos pontos
    for x, y in pontos:
        action.move_by_offset(x, y)
        time.sleep(random.uniform(0.01, 0.03))

    action.perform()


def gerar_curva_bezier(inicio, fim, num_pontos=20):
    """Gera pontos de uma curva Bezier quadrática."""
    # Ponto de controle aleatório (para curvatura)
    controle_x = (inicio[0] + fim[0]) / 2 + random.uniform(-50, 50)
    controle_y = (inicio[1] + fim[1]) / 2 + random.uniform(-50, 50)

    pontos = []
    for t in np.linspace(0, 1, num_pontos):
```

```
        # Fórmula de Bezier quadrática
        x = (1-t)**2 * inicio[0] + 2*(1-t)*t * controle_x + t**2 * fim[0]
        y = (1-t)**2 * inicio[1] + 2*(1-t)*t * controle_y + t**2 * fim[1]
        pontos.append((x, y))

    return pontos
```

10.4.3. Delays Aleatórios

Adicionar delays realistas entre ações.

Python

```python
def delay_entre_acoes(tipo_acao):
    """
    Retorna delay apropriado baseado no tipo de ação.

    Args:
        tipo_acao: 'navegacao', 'click', 'digitacao', 'leitura'
    """
    delays = {
        'navegacao': (2.0, 5.0),      # Aguardar página carregar
        'click': (0.5, 1.5),           # Antes de clicar
        'digitacao': (0.3, 0.8),       # Antes de começar a digitar
        'leitura': (1.0, 3.0),         # Simular leitura de conteúdo
        'entre_mensagens': (30, 120)  # Entre envios de mensagens
    }

    min_delay, max_delay = delays.get(tipo_acao, (1.0, 2.0))
    return random.uniform(min_delay, max_delay)


# Uso
time.sleep(delay_entre_acoes('navegacao'))
```

10.5. Envio de Mensagens

10.5.1. Fluxo Completo de Envio

Python

```python
def enviar_mensagem(driver, perfil_url, mensagem):
    """
    Envia mensagem para um perfil.
```

```
    Returns:
        dict: Resultado do envio
    """
    try:
        # 1. Navegar até perfil
        driver.get(perfil_url)
        time.sleep(delay_entre_acoes('navegacao'))

        # Simular leitura do perfil
        scroll_leve(driver)
        time.sleep(delay_entre_acoes('leitura'))

        # 2. Localizar botão "Enviar mensagem"
        botao_mensagem = localizar_botao_mensagem(driver)

        if not botao_mensagem:
            return {'sucesso': False, 'erro': 'Botão de mensagem não
encontrado'}

        # 3. Mover mouse e clicar
        mover_mouse_natural(driver, botao_mensagem)
        time.sleep(delay_entre_acoes('click'))
        botao_mensagem.click()

        # 4. Aguardar modal abrir
        time.sleep(delay_entre_acoes('navegacao'))

        # 5. Localizar campo de texto
        campo_texto = localizar_campo_texto(driver)

        if not campo_texto:
            return {'sucesso': False, 'erro': 'Campo de texto não
encontrado'}

        # 6. Clicar no campo
        campo_texto.click()
        time.sleep(delay_entre_acoes('digitacao'))

        # 7. Digitar mensagem
        digitar_como_humano(campo_texto, mensagem)

        # 8. Localizar botão enviar
        botao_enviar = localizar_botao_enviar(driver)

        if not botao_enviar:
            return {'sucesso': False, 'erro': 'Botão enviar não encontrado'}
```

```python
        # 9. Enviar
        time.sleep(delay_entre_acoes('click'))
        botao_enviar.click()

        # 10. Aguardar confirmação
        time.sleep(2)

        # 11. Verificar se enviou
        if verificar_envio_sucesso(driver):
            return {'sucesso': True, 'timestamp': time.time()}
        else:
            return {'sucesso': False, 'erro': 'Envio não confirmado'}

    except Exception as e:
        return {'sucesso': False, 'erro': str(e)}


def localizar_botao_mensagem(driver):
    """Localiza botão "Enviar mensagem" (múltiplos seletores)."""
    seletores = [
        "//button[contains(text(), 'Enviar mensagem')]",
        "//button[contains(text(), 'Message')]",
        "//div[@role='button' and contains(text(), 'Enviar mensagem')]"
    ]

    for seletor in seletores:
        try:
            elemento = driver.find_element(By.XPATH, seletor)
            return elemento
        except:
            continue

    return None


def localizar_campo_texto(driver):
    """Localiza campo de texto da mensagem."""
    seletores = [
        "//textarea[@placeholder='Mensagem...']",
        "//textarea[@placeholder='Message...']",
        "//div[@contenteditable='true']"
    ]

    for seletor in seletores:
        try:
            elemento = driver.find_element(By.XPATH, seletor)
            return elemento
        except:
```

```python
            continue

    return None


def localizar_botao_enviar(driver):
    """Localiza botão de enviar mensagem."""
    seletores = [
        "//button[contains(text(), 'Enviar')]",
        "//button[contains(text(), 'Send')]",
        "//button[@type='submit']"
    ]

    for seletor in seletores:
        try:
            elemento = driver.find_element(By.XPATH, seletor)
            return elemento
        except:
            continue

    return None


def verificar_envio_sucesso(driver):
    """Verifica se mensagem foi enviada com sucesso."""
    # Verificar se modal fechou
    try:
        driver.find_element(By.XPATH,
"//textarea[@placeholder='Mensagem...']")
        return False  # Modal ainda aberto
    except:
        return True  # Modal fechou (mensagem enviada)


def scroll_leve(driver):
    """Scroll leve para simular leitura."""
    driver.execute_script("window.scrollBy(0, 300);")
    time.sleep(random.uniform(0.5, 1.5))
    driver.execute_script("window.scrollBy(0, -100);")
```

10.6. Rate Limiting

10.6.1. Implementação de Rate Limiter

Python

```python
import redis
import time

class RateLimiter:
    def __init__(self, redis_client):
        self.redis = redis_client

    def pode_enviar(self, conta_id):
        """
        Verifica se pode enviar mensagem.

        Limites:
        - 20 mensagens por hora
        - 50 mensagens por dia
        """
        # Verificar limite por hora
        chave_hora = f"rate_limit:{conta_id}:hora"
        mensagens_hora = self.redis.get(chave_hora)

        if mensagens_hora and int(mensagens_hora) >= 20:
            return False, "Limite de 20 mensagens por hora atingido"

        # Verificar limite por dia
        chave_dia = f"rate_limit:{conta_id}:dia"
        mensagens_dia = self.redis.get(chave_dia)

        if mensagens_dia and int(mensagens_dia) >= 50:
            return False, "Limite de 50 mensagens por dia atingido"

        return True, None

    def registrar_envio(self, conta_id):
        """Registra envio de mensagem."""
        # Incrementar contador por hora (TTL 1 hora)
        chave_hora = f"rate_limit:{conta_id}:hora"
        self.redis.incr(chave_hora)
        self.redis.expire(chave_hora, 3600)

        # Incrementar contador por dia (TTL 24 horas)
        chave_dia = f"rate_limit:{conta_id}:dia"
        self.redis.incr(chave_dia)
        self.redis.expire(chave_dia, 86400)

    def tempo_ate_proximo_envio(self, conta_id):
        """Retorna tempo em segundos até poder enviar novamente."""
        chave_hora = f"rate_limit:{conta_id}:hora"
        ttl = self.redis.ttl(chave_hora)
```

```
        if ttl > 0:
            return ttl

        return 0
```

10.7. Detecção e Tratamento de Erros

10.7.1. Tipos de Erros

Python

```python
class TipoErro:
    CAPTCHA = "captcha"
    BLOQUEIO_TEMPORARIO = "bloqueio_temporario"
    CONTA_SUSPENSA = "conta_suspensa"
    PERFIL_NAO_ENCONTRADO = "perfil_nao_encontrado"
    PERFIL_PRIVADO = "perfil_privado"
    ERRO_REDE = "erro_rede"
    TIMEOUT = "timeout"
```

10.7.2. Detector de Erros

Python

```python
def detectar_erro(driver):
    """
    Detecta tipo de erro na página.

    Returns:
        str: Tipo de erro ou None
    """
    # Detectar captcha
    if "checkpoint" in driver.current_url or "challenge" in
driver.current_url:
        return TipoErro.CAPTCHA

    # Detectar bloqueio temporário
    if "Tente novamente mais tarde" in driver.page_source:
        return TipoErro.BLOQUEIO_TEMPORARIO

    # Detectar conta suspensa
    if "Sua conta foi suspensa" in driver.page_source:
        return TipoErro.CONTA_SUSPENSA
```

```python
    # Detectar perfil não encontrado
    if "Página não encontrada" in driver.page_source:
        return TipoErro.PERFIL_NAO_ENCONTRADO

    # Detectar perfil privado
    if "Esta conta é privada" in driver.page_source:
        return TipoErro.PERFIL_PRIVADO

    return None
```

10.7.3. Tratamento de Erros

Python

```python
def tratar_erro(tipo_erro, conta, perfil_id):
    """
    Trata erro baseado no tipo.

    Returns:
        dict: Ação a tomar
    """
    if tipo_erro == TipoErro.CAPTCHA:
        # Pausar conta e alertar operador
        return {
            'acao': 'pausar_conta',
            'mensagem': 'Captcha detectado. Resolução manual necessária.',
            'retentar': False
        }

    elif tipo_erro == TipoErro.BLOQUEIO_TEMPORARIO:
        # Pausar conta por 1 hora
        return {
            'acao': 'pausar_conta_temporariamente',
            'duracao_segundos': 3600,
            'retentar': True,
            'retentar_em': 3600
        }

    elif tipo_erro == TipoErro.CONTA_SUSPENSA:
        # Desativar conta permanentemente
        return {
            'acao': 'desativar_conta',
            'mensagem': 'Conta suspensa pelo Instagram',
            'retentar': False
        }
```

```python
    elif tipo_erro == TipoErro.PERFIL_NAO_ENCONTRADO:
        # Marcar perfil como inválido
        return {
            'acao': 'marcar_perfil_invalido',
            'retentar': False
        }

    elif tipo_erro == TipoErro.PERFIL_PRIVADO:
        # Marcar perfil como privado
        return {
            'acao': 'marcar_perfil_privado',
            'retentar': False
        }

    elif tipo_erro == TipoErro.ERRO_REDE:
        # Retentar após 30 segundos
        return {
            'acao': 'retentar',
            'retentar_em': 30
        }

    else:
        # Erro desconhecido
        return {
            'acao': 'logar_erro',
            'retentar': False
        }
```

10.8. Monitoramento e Métricas

10.8.1. Métricas Importantes

Python

```python
metricas_execucao = {
    'mensagens_enviadas': 150,
    'mensagens_falhadas': 12,
    'taxa_sucesso': 0.926,  # 92.6%
    'tempo_medio_envio_segundos': 15.3,
    'captchas_encontrados': 2,
    'bloqueios_temporarios': 1,
    'contas_ativas': 5,
    'contas_pausadas': 1,
```

```
    'mensagens_por_hora': 18.5
}
```

10.8.2. Logging de Envios

Python

```python
import structlog

logger = structlog.get_logger()

def logar_envio(perfil_id, conta_id, resultado):
    """Loga envio de mensagem."""
    logger.info(
        "mensagem_enviada",
        perfil_id=perfil_id,
        conta_id=conta_id,
        sucesso=resultado['sucesso'],
        erro=resultado.get('erro'),
        tempo_segundos=resultado.get('tempo_segundos'),
        timestamp=time.time()
    )
```

10.9. Conclusão do Capítulo

Este capítulo cobriu o **Agente de Execução** em profundidade:

- **Gerenciamento de Sessões:** Sessões persistentes, rotação de contas
- **Simulação de Comportamento Humano:** Digitação natural, movimentos de mouse, delays
- **Envio de Mensagens:** Fluxo completo com múltiplos seletores
- **Rate Limiting:** Limites por hora e dia, implementação com Redis
- **Detecção de Erros:** 7 tipos de erros, tratamento específico
- **Monitoramento:** Métricas e logging

No próximo capítulo, veremos o **Agente de Monitoramento**, que detecta respostas e gerencia conversas.

Próximo Capítulo: Capítulo 11 - Agente de Monitoramento

Referências:

- Instagram Rate Limits: https://www.inflact.com/blog/instagram-limits/
- Selenium Best Practices: https://www.selenium.dev/documentation/test_practices/
- Human Behavior Simulation: https://github.com/ultrafunkamsterdam/undetected-chromedriver